

Decoding the Structure of Language

A Journey into Syntactic Parsing, Chart Parsers, and PCFGs

Based on the lectures of Dr. Chetana Gavankar

Contents

1	Introduction: The Explosion of Ambiguity	2
2	Part 1: The Blueprint of Language (CFGs)	2
2.1	The Phrases	2
3	Part 2: How to Parse (Top-Down vs. Bottom-Up)	3
3.1	The Flaw in Basic Parsing	3
4	Part 3: Chart Parsing (Working Smarter, Not Harder)	3
4.1	The Anatomy of a Chart Parser	4
4.2	Dot Notation: Tracking Progress	4
5	Part 4: The Probabilistic Turn (PCFGs)	4
5.1	Example: "Astronomers saw stars with ears"	5
6	Part 5: Probabilistic CKY Parsing	5
6.1	Chomsky Normal Form (CNF)	5
6.2	The CKY Matrix	6
7	Part 6: Parser Evaluation and Limitations	6
7.1	The Flaws of PCFGs	6
7.2	Parser Evaluation: Precision, Recall, and F1	6
8	Conclusion	7

1 Introduction: The Explosion of Ambiguity

The Tale of the Telescope

Imagine you are a detective, and a witness runs up to you and shouts:

"I saw the man with the telescope!"

As a human, you immediately ask yourself:

1. Did the witness look *through* a telescope to see a man far away?
2. Or was the man carrying a telescope in his hands when the witness saw him?

Both interpretations are grammatically flawless. This is called **Structural Ambiguity**. While humans use common sense to figure out the intended meaning, computers are easily overwhelmed by it.

Ambiguity is explosive. The longer the sentence, the more prepositional phrases (PPs) it has, and the number of possible grammatical trees multiplies exponentially (following a mathematical sequence known as the Catalan numbers).

- "I saw the man with the telescope." → **2 parses**
- "I saw the man on the hill with the telescope." → **5 parses**
- "I saw the man on the hill in Texas with the telescope." → **14 parses**
- "I saw the man on the hill in Texas with the telescope at noon on Monday." → **132 parses!**

Today, we will learn how we teach machines the rules of language (Grammars), how they systematically build trees to understand sentences (Parsing), and how they use statistics to guess the most logical meaning (PCFGs).

2 Part 1: The Blueprint of Language (CFGs)

Before a computer can parse a sentence, it needs a rulebook. We call this rulebook a **Context-Free Grammar (CFG)**.

Key Concept: Formal Definition of a CFG

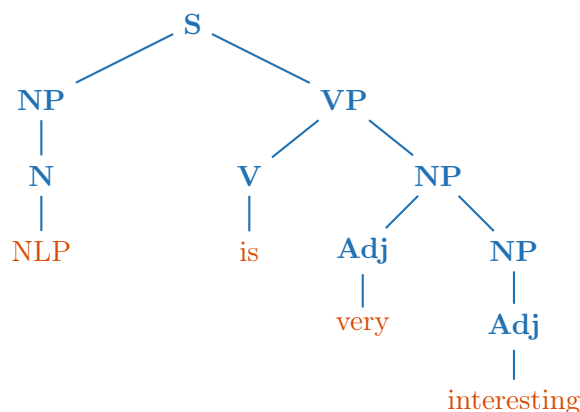
A Context-Free Grammar consists of 4 components (N, Σ, R, S):

1. **N (Non-terminals)**: Abstract structural labels (e.g., S, NP, VP). These are the grammatical categories.
2. **Σ (Terminals)**: The actual words in the language (e.g., "dog", "saw", "the").
3. **R (Rules / Productions)**: Instructions on how to rewrite symbols. They take the form $A \rightarrow \beta$, meaning a single non-terminal A can be rewritten as a sequence β .
4. **S (Start Symbol)**: The designated root of the grammar, usually representing a full Sentence.

2.1 The Phrases

Language is not just a flat list of words; it is organized into hierarchical chunks called phrases. The "head" word determines the type of the phrase:

- **Noun Phrase (NP):** Noun acts as the head. (e.g., "The large can", "NLP")
- **Verb Phrase (VP):** Verb acts as the head. (e.g., "hold the water", "is interesting")
- **Prepositional Phrase (PP):** Preposition acts as the head. (e.g., "with the telescope", "on the hill")



3 Part 2: How to Parse (Top-Down vs. Bottom-Up)

Parsing is the algorithmic process of applying the CFG rules to a sequence of words to build a valid parse tree. There are two main directions to do this.

Analogy Time: The Architect vs. The Builder

Top-Down Parsing (The Architect): You start with the grand vision: "I need to build a Sentence (S)." You know a Sentence is an NP and a VP. You break it down into smaller parts, making guesses, until you eventually match the actual words on the ground.

- *Pro:* Never explores illegal sentence structures.
- *Con:* Wastes time guessing structures that don't match the actual words.

Bottom-Up Parsing (The Builder): You look at the materials on the ground (the words: "The", "dog", "barks"). You see "The" and "dog", and you group them together to form an NP. You group "barks" into a VP. Finally, you combine the NP and VP to form the complete house (S).

- *Pro:* Data-driven; only builds what's actually there.
- *Con:* Might build huge sub-structures that never actually fit into the final blueprint.

3.1 The Flaw in Basic Parsing

Both Top-Down and Bottom-Up approaches suffer from a fatal flaw: **Blind Guessing and Backtracking**. If a parser guesses the wrong rule, it has to throw away all its work and start over. In highly ambiguous sentences, the parser will re-evaluate the exact same phrases hundreds of times. This duplicate work causes exponential delays.

4 Part 3: Chart Parsing (Working Smarter, Not Harder)

To solve the inefficiency of backtracking, we introduce **Chart Parsing**.

Key Concept: The Chart (Dynamic Programming)

A Chart Parser uses a data structure called a **Chart** to store partial results (sub-trees) that it has already figured out. If the parser mathematically proves that "the large can" is a valid Noun Phrase, it saves this in the chart. It will *never* have to parse "the large can" from scratch again, no matter how many times it gets stuck on the rest of the sentence.

4.1 The Anatomy of a Chart Parser

A Bottom-Up Chart Parser maintains key data structures:

1. **The Key:** The current constituent (e.g., a newly found Noun) we are trying to plug into our rules.
2. **The Chart:** The permanent record of fully completed constituents that span substrings of the input.
3. **The Agenda:** A to-do list of newly found, unprocessed constituents.
4. **Active Arcs:** Grammar rules that are *partially* matched and waiting for more pieces.

4.2 Dot Notation: Tracking Progress

To track partial matches in Active Arcs, we use a dot (◦). The dot represents "where we are" in the rule.

- $NP \rightarrow \circ \text{ ART ADJ N}$ (We haven't found anything yet).
- $NP \rightarrow \text{ ART } \circ \text{ ADJ N}$ (We just found an Article! Now waiting for an Adjective).
- $NP \rightarrow \text{ ART ADJ N } \circ$ (We found everything! This rule is complete and goes to the Agenda).

5 Part 4: The Probabilistic Turn (PCFGs)

Standard CFGs and Chart Parsers can tell us *if* a sentence is valid, and they can return *all* possible trees. But remember our 132-parse sentence? A standard parser just dumps 132 trees on our desk. It doesn't know which one makes sense.

We solve this by adding statistics derived from massive Treebanks, creating a **Probabilistic Context-Free Grammar (PCFG)**.

The Math: PCFG Rule Probabilities

In a PCFG, every rule is assigned a conditional probability: $P(A \rightarrow \beta)$.

Crucial Constraint: For any non-terminal A , the sum of the probabilities of all its possible expansions must equal exactly 1.

$$\sum_{\beta} P(A \rightarrow \beta) = 1$$

Probability of a Parse Tree ($P(T)$): The total probability of an entire parse tree is simply the **product** of the probabilities of all the individual rules used to build it:

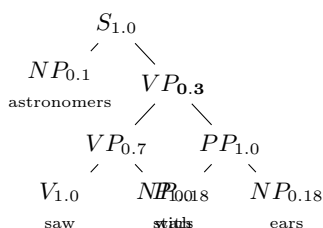
$$P(T) = \prod_{i=1}^k P(r_i)$$

If we have two competing trees, the machine calculates $P(T_1)$ and $P(T_2)$ and chooses the tree with the higher score!

5.1 Example: "Astronomers saw stars with ears"

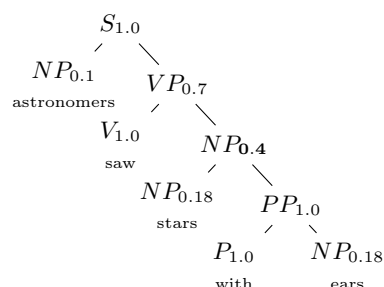
Consider the ambiguity: Did the stars have ears, or did the astronomers use ears to see?

Tree 1: PP attached to VP
(Using ears as an instrument)



$$P(T_1) = 1.0 \times 0.1 \times \mathbf{0.3} \times 0.7 \cdots = \mathbf{0.00068}$$

Tree 2: PP attached to NP
(The stars possess ears)



$$P(T_2) = 1.0 \times 0.1 \times 0.7 \times 1.0 \times \mathbf{0.4} \cdots = \mathbf{0.00090}$$

Because $0.00090 > 0.00068$, the PCFG concludes that **Tree 2** is more mathematically likely based on its training data!

6 Part 5: Probabilistic CKY Parsing

To efficiently compute these probabilities for long sentences, we use the **CKY (Cocke-Kasami-Younger) Algorithm**. CKY is a dynamic programming algorithm that builds a triangular table from the bottom up.

6.1 Chomsky Normal Form (CNF)

CKY requires the grammar to be in a very specific format called **Chomsky Normal Form**. In CNF, every rule must look like one of two things:

1. $A \rightarrow B C$ (A non-terminal expands into exactly **two** non-terminals).
2. $A \rightarrow a$ (A non-terminal expands into exactly **one** terminal word).

Converting to CNF:

- **Mix of Terminals and Non-Terminals:** ($NP \rightarrow \text{the Nominal}$)
Fix: Introduce a dummy rule. $Det \rightarrow \text{the}$, then rewrite as $NP \rightarrow Det \text{ Nominal}$.
- **Unit Productions:** ($NP \rightarrow \text{Nominal}$)
Fix: If $Nominal \rightarrow \text{Noun PP}$, just rewrite it directly as $NP \rightarrow \text{Noun PP}$.
- **Too Long:** ($NP \rightarrow Det \text{ Noun PP}$)
Fix: Chain them. Create an intermediate dummy, like X . $NP \rightarrow Det X$, and $X \rightarrow \text{Noun PP}$.

6.2 The CKY Matrix

Let's parse: "The flight includes a meal." (5 words). We create a grid where the columns and rows represent the spans of words. We combine adjacent cells to form bigger phrases, tracking the probabilities as we go.

Span	1 (The)	2 (flight)	3 (in-cludes)	4 (a)	5 (meal)
0	Det (0.4)	NP (0.3 × ...)	-	-	S (Max Prob)
1		N (0.02)	-	-	-
2			V (0.05)	-	VP (0.2 × ...)
3				Det (0.4)	NP (0.3 × ...)
4					N (0.01)

How did we get the probability for NP in cell (0,2)? We looked at the cell for "The" (Det: 0.4) and "flight" (N: 0.02). The rule is $NP \rightarrow Det N$ with a probability of 0.3.

$$P(NP) = 0.4 \times 0.02 \times 0.3 = \mathbf{0.0024}$$

The CKY algorithm fills this table diagonally until it reaches the top-right corner, representing the start symbol **S** spanning the entire sentence!

7 Part 6: Parser Evaluation and Limitations

7.1 The Flaws of PCFGs

While powerful, standard PCFGs have distinct weaknesses:

- **No Lexicalization:** They treat all words equally based on their tags. $VP \rightarrow V NP NP$ is evaluated the same way whether the verb is "give" (makes sense: "give him a book") or "sleep" (doesn't make sense: "sleep him a book"). PCFGs miss these lexical dependencies.
- **No Structural Context:** PCFGs assume rules are completely context-free. But in reality, an NP is more likely to expand into a pronoun if it is the subject of a sentence, compared to when it is the object.

7.2 Parser Evaluation: Precision, Recall, and F1

How do we know if our parser is good? We compare its output (Candidate Brackets) against a human-annotated treebank (Gold Standard Brackets). We evaluate based on **Constituents** (e.g., $NP - (0 : 2)$).

The Math: Evaluation Metrics

- **Labeled Precision (LP):** Out of all the constituents the machine *predicted*, what percentage were actually correct?

$$\text{Precision} = \frac{\text{Correct Predicted Constituents}}{\text{Total Predicted Constituents}}$$

- **Labeled Recall (LR):** Out of all the constituents in the *Gold Standard*, what

percentage did the machine successfully find?

$$\text{Recall} = \frac{\text{Correct Predicted Constituents}}{\text{Total Gold Standard Constituents}}$$

- **F1-Measure:** The harmonic mean, balancing Precision and Recall.

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why both matter: If a parser wildly guesses 100 phrases for a short sentence, it might accidentally find all 6 true Gold Standard phrases. Its Recall would be 100%, but its Precision would be a terrible 6%. The F1-Measure ensures the parser is both accurate and comprehensive.

8 Conclusion

Parsing takes language from a flat string of characters and transforms it into a structured, logical map of meaning.

- **CFGs** provide the structural rules.
- **Chart Parsing** uses dynamic programming (caching) to prevent duplicate work during the search.
- **PCFGs** add probabilities to mathematically resolve structural ambiguities.
- **The CKY Algorithm** elegantly calculates the highest-probability tree using Chomsky Normal Form.